COMPUTER NETWORKS LAB (Course Code: V23CSL09)

B. TECH V Semester (2024-2028 Batch)

Academic Year:2026-27



SRI VASAVI ENGINEERING COLLEGE(Autonomous)

Pedatadepalli, Tadepalligudem-534101

Department of Computer Science & Engineering (Accredited by NBA)

SRI VASAVI ENGINEERING COLLEGE(Autonomous)

PEDATADEPALLI, TADEPALLIGUDEM.



CERTIFICATE

This is to certify that this a bonafide record of practical work done in **Computer Networks Lab (Course** Code: **V23CSL09)** by Mr: **J.Akhil** bearing rollno:**24A81A0580** of **CSE** Branch V Semester during the academic year 2026-2027.

No of Experiments Done:

Faculty In Charge of the Laboratory

Head of the Department

Examiner 1

Examiner 2

INDEX

S.NO	DATE	NAME OF THE EXPERIMENT	P.NO	SIGNATURE
1.		Study of Network devices in detail and connect the computers in Local Area Network.		
2.		Develop a Program to implement the data link layer framing methods such as: i) Character stuffing ii) Bit stuffing.		
3.		Develop a Program to implement data link layer framing method checksum.		
4.		Develop a program for Hamming Code generation for error detection and correction.		
5.		Develop a Program to implement on a data set of characters the three CRC polynomials – CRC 12, CRC 16 and CRC CCIP.		
6.		Develop a Program to implement Sliding Window Protocol for Go-Back-N.		
7.		Develop a program to implement Sliding Window Protocol for Selective Repeat.		
8.		Develop a Program to implement Stop and Wait Protocol.		
9.		Develop a program for congestion control using Leaky Bucket Algorithm.		
10.		Develop a Program to implement Dijkstra's algorithm to compute the Shortest Path through a graph.		
11.		Develop a Program to implement Distance Vector Routing Algorithm by obtaining routing table at each node (Take an example subnet graph with weights indicating delay between nodes).		
12.		Develop a Program to implement Broadcast Tree by taking subnet of hosts.		
13.		**Wireshark** i) Packet Capture Using Wireshark ii) Starting Wireshark iii) Viewing Captured Traffic iv) Analysis and Statistics & Filters.		

14.		Discover active hosts, open ports and running services in a network using Nmap.		
15.		Find Operating System in a host using Nmap.		
16.		**Do the following using NS2 Simulator:** i) NS2 Simulator – Introduction ii) Simulate to Find the Number of Packets Dropped iii) Simulate to Find the Number of Packets Dropped by TCP/UDP iv) Simulate to Find the Number of Packets Dropped due to Congestion v) Simulate to Compare Data Rate & Throughput.		

EXPERIMENT – 1

Study of Network Devices in detail and connect the computers in Local Area Network.

Network Devices:

Hub

- A hub is a basic networking device that connects multiple computers in a network and transmits data to all connected devices, regardless of the intended recipients.
- It operates at the OSI layer 1- physical layer
- It does not filter or manage traffic, leading to network collisions.

Switch

- A switch is a networking device that connects multiple devices within a LAN and forwards data only to the device it is intended for using MAC addresses.
- It operates at the layer 2- Data Link Layer.
- It significantly reduces network collisions compared to a hub.

Router

- A router connects different networks together(Like LAN to WAN) and directs data packets based on IP addresses.
- It works at the network layer – Layer 3 and is essential for internet.

Layer 3 Switch

- A Layer 3 switch combines the functionality of a switch and a router.
- It performs high speed switching within a network(like a switch) and routing between different subnets(like a router) using IP addresses.
- It is commonly used in large networks to improve performance and manage traffic efficiently.

Repeater

- A repeater amplifies or regenerates signals to extend the range of a network.
- It is used to overcome distance limitations in wired or wireless communications by boosting weak signals.

.

IDS (Intrusion Detection System)

- An IDS monitors network traffic for suspicious activity and potential threats.
- It alerts administrators of possible security breaches but does not take action to prevent them**.**

Application Gateway

- It is also known as an Application Layer Gateway(ALG).
- It filters traffic fro specific applications like FTP, HTTP,etc.
- It acts as a proxy and provides security and protocol awareness at the application layer.

Gateway

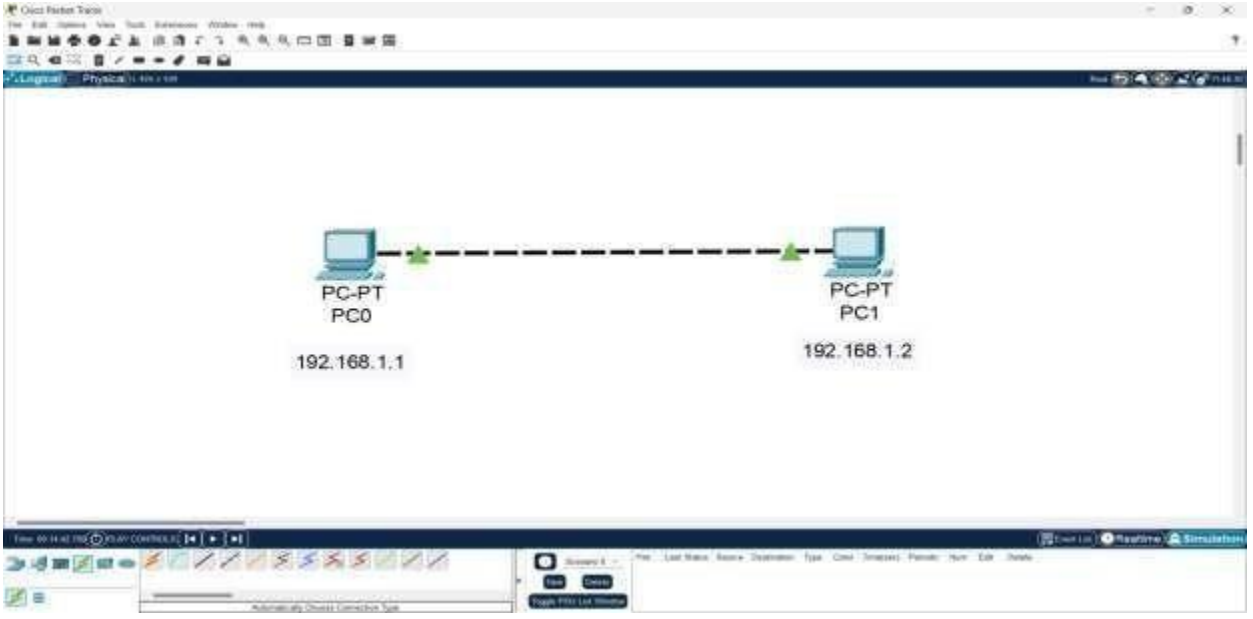
- A gateway is a device that connects networks using different protocols.
- It acts as a translator between networks, often used for connecting enterprise networks to external networks like the INTERNET.

K-Trace

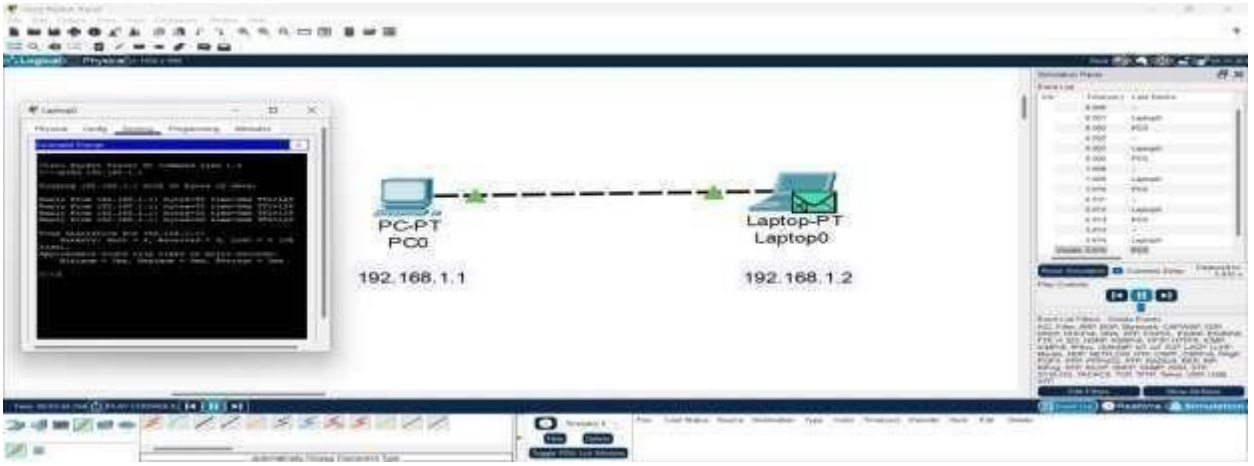
- K-Trace is a network diagnostic tool used in simulations like Cisco Packet Tracer.
- It helps trace the path a packet takes from source to destination, identifying hops and delays.

Connection of computers in Local Area Network

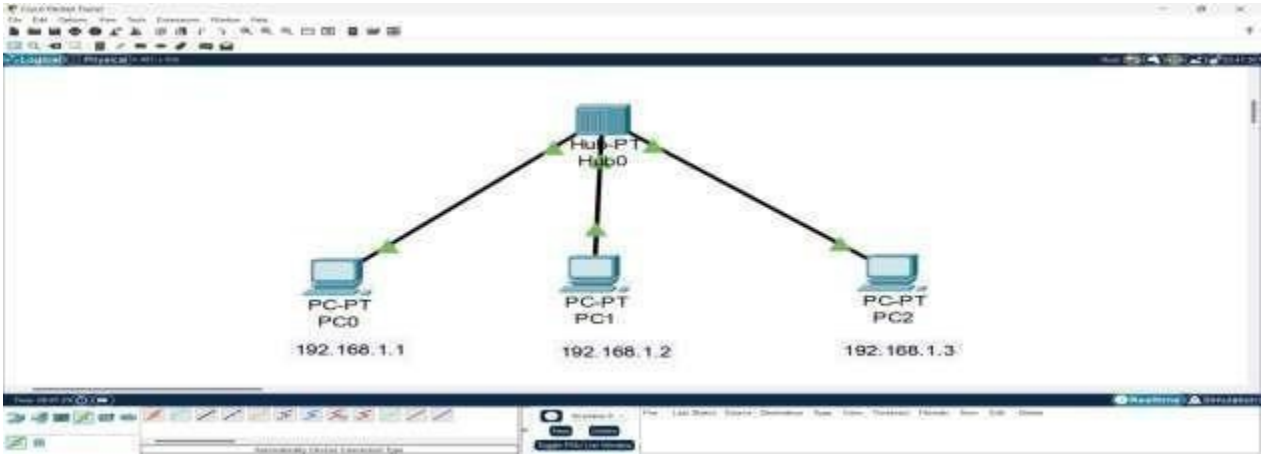
Connectivity between two PC's



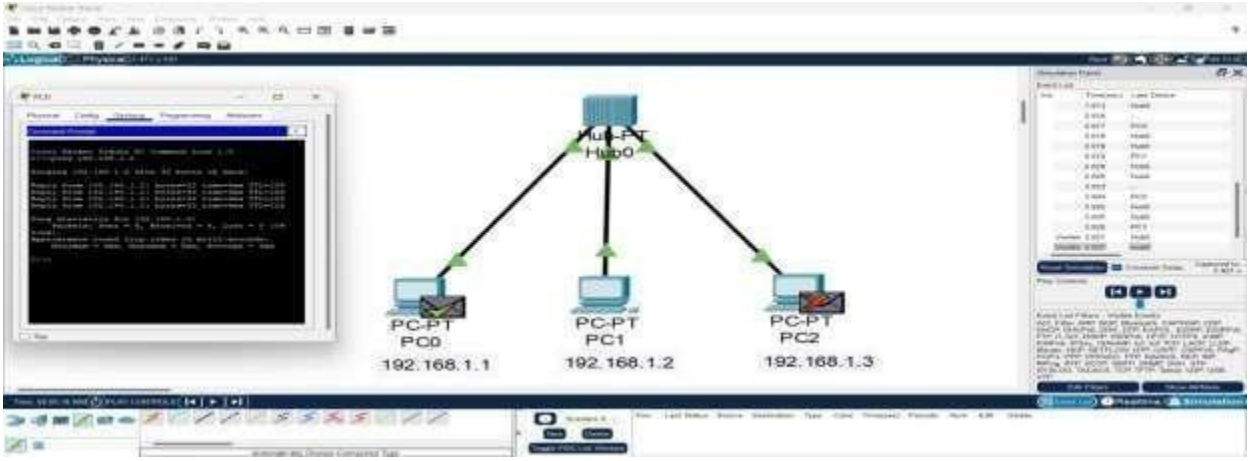
- The packet was sent from PC1 to PC0 through the direct connection.
- The packet passed down the OSI layers, was encapsulated with the destination MAC and IP addresses, and transmitted over the cable.
- PC0 received the packet, decapsulated it and replied with an echo reply using the same process using the same process in reverse.



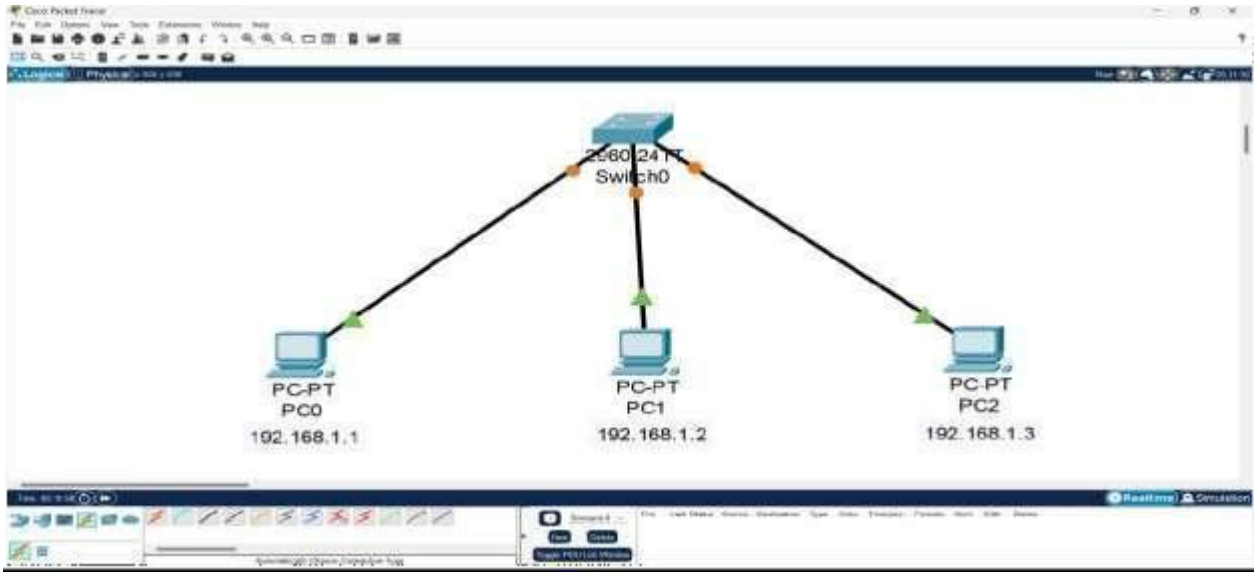
Connecting 3 PC's with a Hub

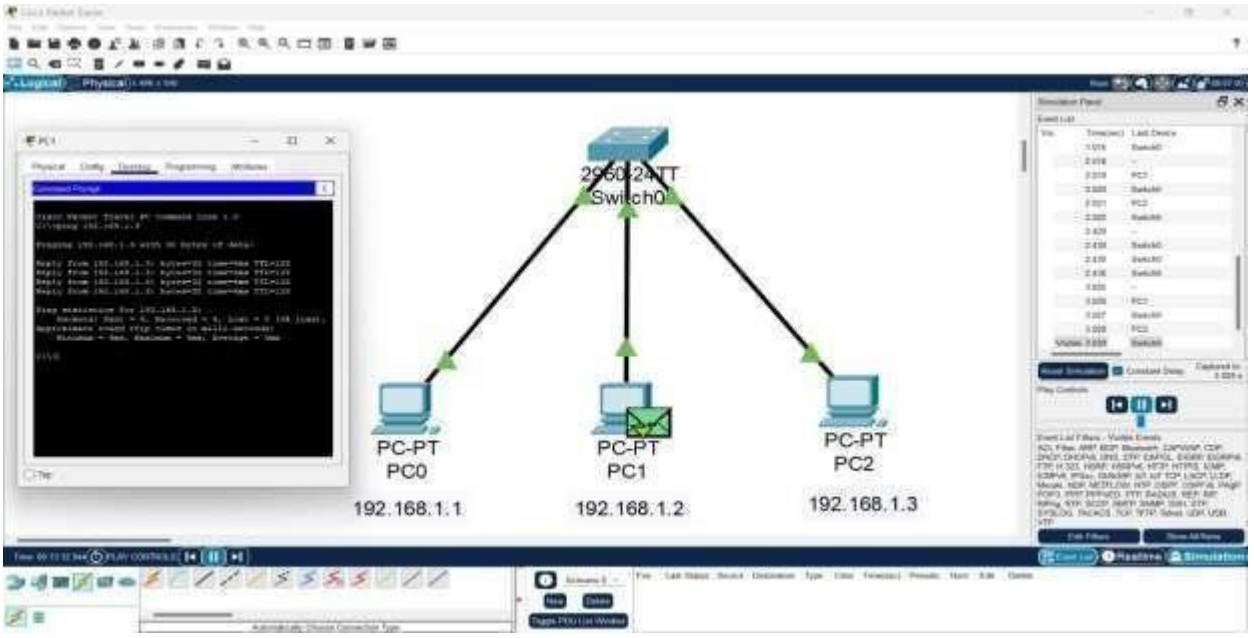


- The packet was sent from PC0 to PC1 through the hub.
- The hub broadcasted the packet to all connected devices, including PC2.
- Only PC1, recognizing it's IP address (192.168.1.2), accepted and processed the packet.
- PC1 then replied to PC0 using the same broadcast path.



Connecting 3 PC's with a switch





- The packet was sent from PC1 to PC2 through the switch.
- The switch used its MAC address table to forward the packet only to PC2, avoiding unnecessary broadcasting to PC0.
- PC2 received the packet and replied directly to PC1 via the switch.

EXPERIMENT-2

Develop a program to implement the data link layer framing methods such as

i. Character Stuffing

ii. Bit Stuffing

Character Stuffing

Description:

Character Stuffing is a data link layer framing technique used to identify the beginning and end of a frame when data is transmitted between two devices.
In character stuffing, **special characters** are used to mark the **start and end of a frame**. If the same special character occurs within the actual data, an **escape character (ESC)** is inserted before it. This prevents the receiver from confusing the data character with a frame boundary.

Program:

```
#include <stdio.h>

#include<string.h>

int main()

{

   char data[100], stuffed[200];

   int i, j = 0, count = 0;

   printf("Enter the data: ");

   scanf("%s", data);

   for (i = 0; i < strlen(data); i++)

   {

      stuffed[j++] = data[i];

      if (data[i] == '1')

      {

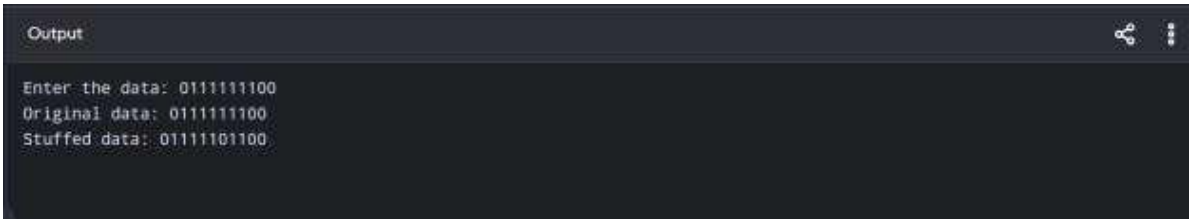
         count++;

        if (count == 5)

         {
```

```
            stuffed[j++] = '0';
            count = 0;
        }
      }
      else
    {
        count = 0;
      }
   }
  stuffed[j] = '\0';
  printf("Original data: %s\n", data);
  printf("Stuffed data: %s\n", stuffed);
  return 0;
}
```

Output:



Bit Stuffing

Description:

Bit Stuffing is a data link layer framing technique used to identify the **beginning and end of a frame** during data transmission.
In bit stuffing, a special bit pattern called a **flag** is used to indicate the start and end of a frame. In HDLC, the flag is usually:
01111110
To prevent this flag pattern from occurring accidentally inside the data, the sender **inserts a 0 bit after every sequence of five consecutive 1s** in the data.

Program:

```
#include <stdio.h>

#include<string.h>

int main()

{

    char data[100], stuffed[200], destuffed[100];

    char flag, esc;

    int i, j = 0, k = 0;

    printf("Enter the data: ");

    scanf("%s", data);

    printf("Enter the flag character: ");

    scanf(" %c", &flag);

    printf("Enter the escape character: ");

    scanf(" %c", &esc);

    /* Character Stuffing */

    stuffed[j++] = flag;

    for (i = 0; i < strlen(data); i++)

    {

        if (data[i] == flag || data[i] == esc)

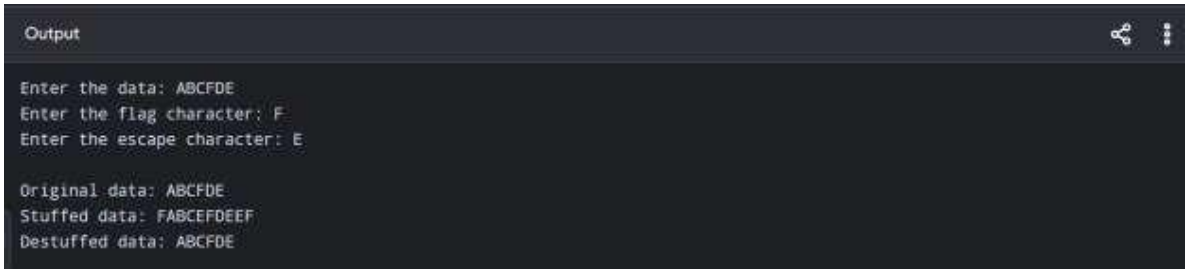
        {

            stuffed[j++] = esc;

        }
```

```
        stuffed[j++] = data[i];
    }
    stuffed[j++] = flag;
    stuffed[j] = '\0';
    printf("\nOriginal data: %s", data);
    printf("\nStuffed data: %s", stuffed);
    /* Character Destuffing */
    k = 0;
    for (i = 1; i < strlen(stuffed) - 1; i++)
    {
        if (stuffed[i] == esc)
        {
            i++;
        }
        destuffed[k++] = stuffed[i];
    }
    destuffed[k] = '\0';
    printf("\nDestuffed data: %s\n", destuffed);
    return 0;
}
```

Output:



EXPERIMENT-3

Develop a program to implement data link layer framing method checksum.

Description:

Checksum is an error-detection technique used in the data link layer.
It divides the data into fixed-size blocks and adds them using binary addition.
The complement of the final sum is calculated as the checksum.
The checksum is sent along with the original data to the receiver.
The receiver performs the same calculation to check for errors.
If the result is correct, the data is accepted; otherwise, an error is detected.

Program:

```
#include <stdio.h>

#include <string.h>

int binaryToDecimal(char bin[])

{

   int i, value = 0;

   for (i = 0; bin[i] != '\0'; i++)

   {

      value = value * 2 + (bin[i] - '0');

   }

   return value;

}

void decimalToBinary(int value, char bin[], int bits)

{

   int i;

   bin[bits] = '\0';

   for (i = bits - 1; i >= 0; i--)

   {

      bin[i] = (value % 2) + '0';

      value = value / 2;

   }

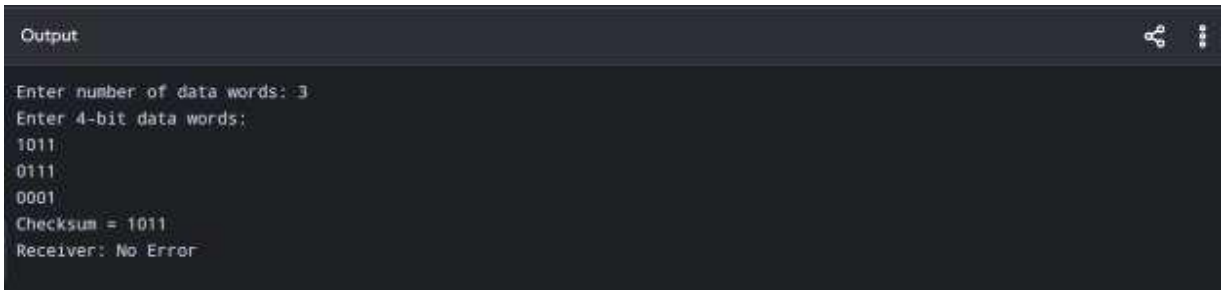
}
```

```
int main()

{
char data[20][20], checksum[20]; int i, n, sum = 0;
printf("Enter number of data words: ");
scanf("%d", &n);
printf("Enter 4-bit data words:\n"); for
(i = 0; i < n; i++)
{
  scanf("%s", data[i]);
   sum = sum + binaryToDecimal(data[i]);
}
/* Wraparound carry */
while (sum > 15)
   {
      sum = (sum & 15) + (sum >> 4);
   }
   /* 1's complement */
   sum = 15 - sum;
   decimalToBinary(sum, checksum, 4);
   printf("Checksum = %s\n", checksum);
   /* Receiverside */
   sum = 0;
   for (i = 0; i < n; i++)
   {
      sum = sum + binaryToDecimal(data[i]);
   }
   sum = sum + binaryToDecimal(checksum);
   /* Wraparound carry */
   while (sum > 15)
```

```
    {
        sum = (sum & 15) + (sum >> 4);
    }
    if (sum == 15)
    {
        printf("Receiver: No Error\n");
    }
    else
    {
        printf("Receiver: Error Detected\n");
    }
    return 0;
}
```

Output:



EXPERIMENT-4

Develop a program for hamming code generation for error detection and correction.

Description:

Hamming Code is an error-detection and error-correction technique used in data communication.
It adds **parity bits** to the original data bits at specific positions.
The parity bits are calculated based on the positions of the data bits.
At the receiver, parity bits are checked to identify the position of an error.
Hamming Code can **detect and correct a single-bit error** in the transmitted data.
Thus, it provides reliable communication by correcting errors without retransmission.

Program:

```
#include <stdio.h>

int main()

{

    int  d[8],  r[8];

    int p1, p2, p4;

    int error, i;

    printf("Enter 4 data bits: ");

    scanf("%d %d %d %d", &d[1], &d[2], &d[3], &d[4]);

    /* Calculate parity bits */

    p1 = d[1] ^ d[2] ^ d[4];

    p2 = d[1] ^ d[3] ^ d[4];

    p4 = d[2] ^ d[3] ^ d[4];

    /* Construct Hamming code */

    r[1] = p1;

    r[2] = p2;

    r[3] = d[1];

    r[4] = p4;

    r[5] = d[2];

    r[6] = d[3];

    r[7] = d[4];
```

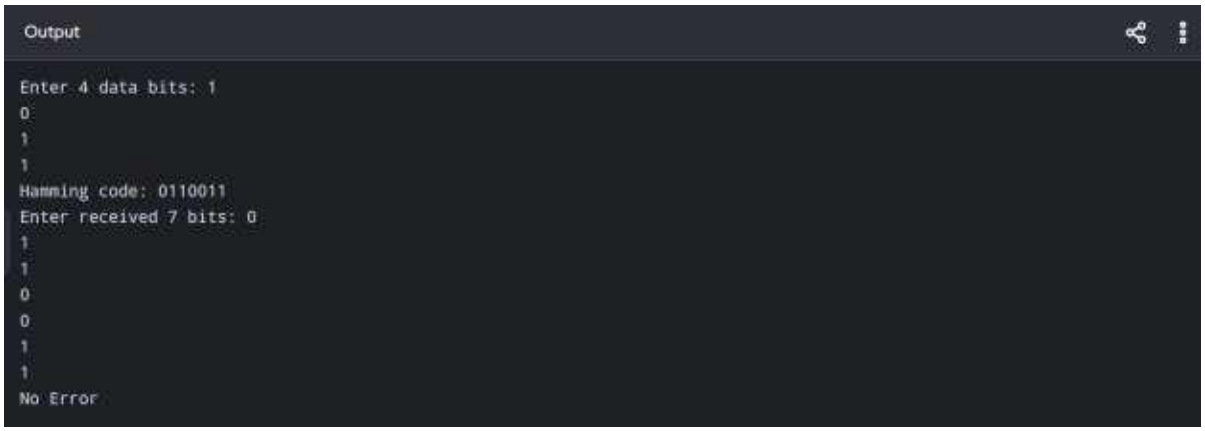
```
printf("Hamming code: ");
        for (i = 1; i <= 7; i++)
{
   printf("%d",r[i]);
  printf("\n");
/* Enter received 7-bit code */
printf("Enter received 7 bits: ");
for (i = 1; i <= 7; i++)
{
   scanf("%d", &r[i]);
}
/* Calculate error position */
p1 = r[1] ^ r[3] ^ r[5] ^ r[7];
p2 = r[2] ^ r[3] ^ r[6] ^ r[7];
p4 = r[4] ^ r[5] ^ r[6] ^ r[7];
error = p1 * 1 + p2 * 2 + p4 * 4;
if (error == 0)
{
   printf("No Error\n");
}
else
{
   printf("Error at position: %d\n", error);
   /* Correct the error */
   r[error] = r[error] ^ 1;
   printf("Corrected code: ");
   for (i = 1; i <= 7; i++)
   {
      printf("%d", r[i]);
```

```
        }

        printf("\n");
    }

    return 0;
}
```

Output:



EXPERIMENT-5

Develop a program to implement on a data set of characters the three CRC polynomials – CRC 12, CRC 16, CRC-CCITT.

Description:

CRC (Cyclic Redundancy Check) is an error-detection technique used in data communication.
It treats the data as a binary sequence and performs polynomial division.
The remainder obtained from the division is called the **CRC value**.
The CRC value is appended to the original data before transmission.
The receiver performs the same polynomial division to check for errors.
The program implements **CRC-12, CRC-16, and CRC-CCITT** polynomials to detect transmission errors.

Program:

```
#include <stdio.h>

#include <string.h>

void crc(char data[], char poly[], char result[])

{

   int n = strlen(data);

   int m = strlen(poly);

   char temp[200];

   int i, j;

   strcpy(temp, data);

   for (i = 0; i <= n - m; i++)

   {

      if (temp[i] == '1')

      {

         for (j = 0; j < m; j++)

            temp[i + j] = (temp[i + j] == poly[j]) ? '0' : '1';

      }

   }

   strcpy(result, temp + n - m + 1);

}
```

```
void calculateCRC(char data[], char poly[], char name[])
{
    char appended[200], remainder[200];
    strcpy(appended, data);
    for (int i = 0; i < strlen(poly) - 1; i++)
    strcat(appended, "0");
    crc(appended, poly, remainder);
    printf("\n%s:\n", name);
    printf("Generator : %s\n", poly);
    printf("CRC       : %s\n", remainder);
    printf("Codeword : %s|%s\n", data, remainder);
}

int main()
{
    char data[100];
    printf("Enter data bits: ");
    scanf("%s", data);
    calculateCRC(data, "1100000001111", "CRC-12");
    calculateCRC(data, "11000000000000101", "CRC-16");
    calculateCRC(data, "10001000000100001", "CRC-CCITT");
    return 0;
}
```

Output:


Output
Enter data bits: 1101011011
CRC-12:
Generator : 1100000001111
CRC : 110001100100
Codeword : 1101011011|110001100100
CRC-16:
Generator : 11000000000000101
CRC : 1000101111011001
Codeword : 1101011011|1000101111011001
CRC-CCITT:
Generator : 10001000000100001
CRC : 1011111011001101
Codeword : 1101011011|1011111011001101

EXPERIMENT-6

Develop a program to implement Sliding window protocol for Goback N.

Description:

Go-Back-N is a **sliding window protocol** used for reliable data transmission.
It allows the sender to transmit multiple frames before receiving acknowledgements.
The receiver sends an acknowledgement for correctly received frames.
If a frame is lost or corrupted, the sender retransmits that frame and all subsequent frames.
The window then moves forward as acknowledgements are received.
Thus, Go-Back-N provides **error control and efficient data transmission**.

Program:

```
#include <stdio.h>

int main()

{

   int n, window, i, start;

   printf("Enter number of frames: ");

   scanf("%d", &n);

   printf("Enter window size: ");

   scanf("%d", &window);

   start = 0;

   while (start < n)

   {

      printf("\nSending frames:  ");

      for (i = start; i < start + window && i < n; i++)

         printf("%d ", i);

      printf("\nEnter the frame number that was lost (-1 if none): ");

      int lost;

      scanf("%d",&lost);

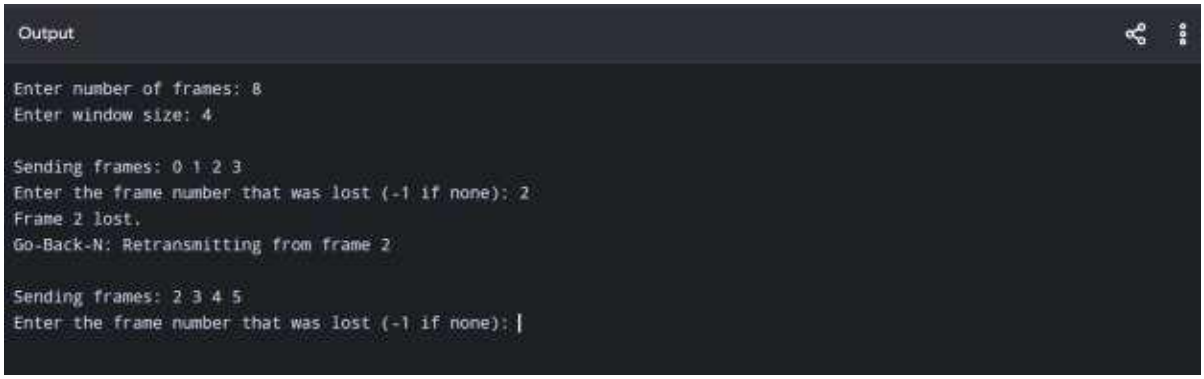
      if (lost == -1)

      {

         printf("All frames acknowledged.\n");
```

```
            start += window;
        }
        else
        {
            printf("Frame %d lost.\n", lost);
            printf("Go-Back-N: Retransmitting from frame %d\n", lost);
            start = lost;
        }
    }
    printf("\nAll frames transmitted successfully.\n");
    return 0;
}
```

Output:



EXPERIMENT-7

Develop a program to implement Sliding window protocol for Selective repeat.

Description:

Selective Repeat is a **sliding window protocol** used for reliable data transmission.
It allows the sender to transmit multiple frames before receiving acknowledgements.
The receiver can accept and store frames that arrive out of order.
If a frame is lost or corrupted, only that particular frame is retransmitted.
The sender retransmits the missing frame after receiving a negative acknowledgement or timeout.
Thus, Selective Repeat provides **efficient error control and reliable data transmission**.

Program:

```
#include <stdio.h>

int main()

{

   int n, window;

   int ack[100] = {0};

   int start = 0, i, lost;

   printf("Enter number of frames: ");

   scanf("%d", &n);

   printf("Enter window size: ");

   scanf("%d", &window);

   while (start < n)

   {

      printf("\nSending frames:  ");

      for (i = start; i < start + window && i < n; i++)

      {

         if (!ack[i])

            printf("%d ", i);

      }

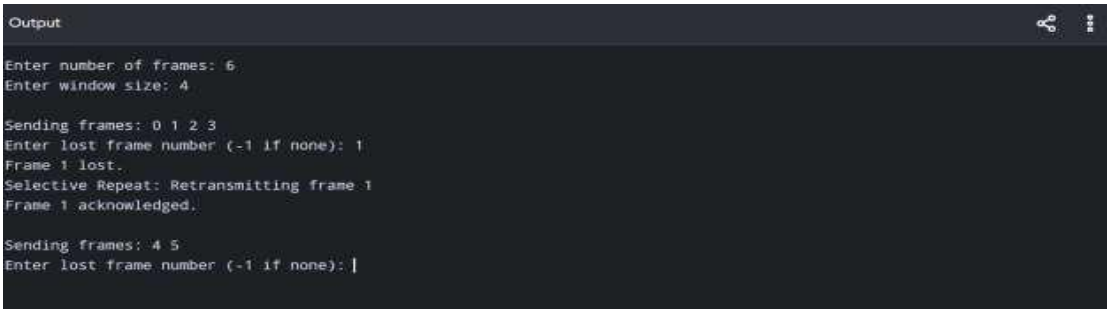
      printf("\nEnter lost frame number (-1 if none): ");

      scanf("%d", &lost);

      if (lost == -1)
```

```
        {
for (i = start; i < start + window && i < n; i++)
            {
                ack[i] = 1;
            }
            printf("All frames acknowledged.\n");
        }
        else
        {
            printf("Frame %d lost.\n", lost);
            for (i = start; i < start + window && i < n; i++)
            {
                if (i != lost)
                    ack[i] = 1;
            }
            printf("Selective Repeat: Retransmitting frame %d\n", lost);
            ack[lost] = 1;
            printf("Frame %d acknowledged.\n", lost);
        }
        while (start < n && ack[start])
            start++;
    }
    printf("\nAll frames transmitted successfully.\n");
    return 0;
}
```

Output:


Output
Enter number of frames: 6
Enter window size: 4
Sending frames: 0 1 2 3
Enter lost frame number (-1 if none): 1
Frame 1 lost.
Selective Repeat: Retransmitting frame 1
Frame 1 acknowledged.
Sending frames: 4 5
Enter lost frame number (-1 if none):

EXPERIMENT-8

Develop a program to implement Stop and Wait Protocol.

Description:

Stop and Wait is a flow control and error control protocol used for reliable data transmission.
The sender sends one frame at a time and waits for an acknowledgement.
The receiver sends an ACK when the frame is received correctly.
If the ACK is not received within a given time, the sender retransmits the frame.
The sender sends the next frame only after receiving the acknowledgement.
Thus, Stop and Wait ensures reliable and error-free transmission of data.

Program:

```
#include <stdio.h>

int main()

{

   int n, i, lost;

   printf("Enter number of frames: ");

   scanf("%d", &n);

   for (i = 0; i < n; i++)

   {

      printf("\nSending frame %d...\n", i);

      printf("Was frame %d lost? (1-Yes, 0-No): ", i);

      scanf("%d", &lost);

      if (lost == 1)

      {

         printf("Frame %d lost.\n", i);

         printf("Retransmitting frame %d...\n", i);

      }

      printf("Frame %d received successfully.\n", i);

      printf("ACK %d received.\n", i);

   }

   printf("\nAll frames transmitted successfully.\n");
```

```
    return 0;
}
```

Output:



EXPERIMENT-9

Develop a program for congestion control using leaky bucket algorithm.

Description:

The Leaky Bucket algorithm is a congestion control mechanism used in computer networks to manage data transmission. It stores incoming packets in a bucket of fixed capacity and sends them at a constant rate. If the bucket has enough space, the packets are accepted; otherwise, they are rejected when the bucket overflows. This process helps regulate network traffic, prevent congestion, and ensure smooth and reliable data transmission.

Program

```
#include <stdio.h>

int main()

{

   int bucket_size, output_rate, n;

   int packet_size, stored = 0;

   printf("Enter bucket size: ");

   scanf("%d", &bucket_size);

   printf("Enter output rate: ");

   scanf("%d", &output_rate);


   printf("Enter number of packets: ");

   scanf("%d", &n);

   for (int i = 1; i <= n; i++)

   {

      printf("\nEnter packet size %d: ", i);

      scanf("%d", &packet_size);

      if (packet_size <= bucket_size - stored)

      {

         stored += packet_size;

         printf("Packet accepted\n");
```

```
            printf("Bytes in bucket: %d\n", stored);
        }
        else
        {
            printf("Packet rejected (Bucket overflow)\n");
        }
        if (stored > 0)
        {
            if (stored >= output_rate)
                stored -= output_rate;
            else
                stored = 0;
            printf("Bytes remaining after transmission: %d\n",
                  stored);
        }
    }
    while (stored > 0)
    {
        if (stored >= output_rate)
            stored -= output_rate;
        else
            stored = 0;
        printf("Bytes remaining after transmission: %d\n",
              stored);
    }
    return 0;
}
```

Output:

```
Enter bucket size: 10
Enter output rate: 4
Enter number of packets: 3

Enter packet size 1: 5
Packet accepted
Bytes in bucket: 5
Bytes remaining after transmission: 1

Enter packet size 2: 8
Packet accepted
Bytes in bucket: 9
Bytes remaining after transmission: 5

Enter packet size 3: 7
Packet rejected (Bucket overflow)
Bytes remaining after transmission: 1
Bytes remaining after transmission: 0
```

EXPERIMENT-10

Develop a program to implement dijkstras algorithm to compute the shortest path through a graph.

Description:

Dijkstra's algorithm is used to find the shortest path from a source vertex to all other vertices in a weighted graph. It works by selecting the unvisited vertex with the minimum distance and updating the distances of its neighboring vertices. This process continues until all reachable vertices are visited. The algorithm helps determine the minimum distance between vertices and is widely used in computer networks, GPS navigation, and route planning. It works only with non-negative edge weights.

Program

```
#include <stdio.h>

#define INF 9999

int main() {
   int n, a[10][10], d[10], v[10] = {0};
   int i, j, s, u, min;
   printf("Enter number of vertices: ");
   scanf("%d", &n);
   printf("Enter adjacency matrix:\n");
   for (i = 0; i < n; i++)
      for (j = 0; j < n; j++) {
         scanf("%d", &a[i][j]);
         if (i != j && a[i][j] == 0)
            a[i][j] = INF;


   printf("Enter source vertex: ");
   scanf("%d", &s);
   for (i = 0; i < n; i++)
      d[i] = a[s][i];
   d[s] = 0;
   for (i = 0; i < n; i++) {
```

```
        min = INF;
        u = -1;
        for (j = 0; j < n; j++)
            if (!v[j] && d[j] < min)
                min = d[j], u = j
        if (u == -1) break;
        v[u] = 1;
        for (j = 0; j < n; j++)
            if (!v[j] && d[u] + a[u][j] < d[j])
                d[j] = d[u] + a[u][j];
    }
    printf("Shortest distances:\n");
    for (i = 0; i < n; i++)
        if (d[i] == INF)
            printf("%d -> Unreachable\n", i);
        else
            printf("%d -> %d\n", i, d[i]);
    return 0;
}
```

Output:

```
Enter number of vertices: 4
Enter adjacency matrix:
0 2 5 0
2 0 1 4
5 1 0 2
0 4 2 0
Enter source vertex: 0
Shortest distances:
0 -> 0
1 -> 2
2 -> 3
3 -> 5
```

EXPERIMENT-11

Develop a program to implement distance vector routing algorithm by obtaining routing table at each node(Take an example subset graph with weights including delay between nodes).

Description:

The Distance Vector Routing algorithm is used to determine the shortest paths between nodes in a computer network. Each node maintains a routing table containing the minimum distance to every destination. Nodes exchange routing information with their neighboring nodes and update their tables based on the received distances and link costs. This process continues until the routing tables converge. The algorithm helps find efficient routes and minimize communication delays in a network.

Program

```
#include <stdio.h>

#define INF 99

int main() {

   int n, cost[10][10], dist[10][10], i, j, k

   printf("Enter number of nodes: ");

   scanf("%d", &n);

   printf("Enter cost matrix (0 for no link):\n");

   for (i = 0; i < n; i++)

      for (j = 0; j < n; j++) {

         scanf("%d", &cost[i][j]);

         if (i != j && cost[i][j] == 0)

            cost[i][j] = INF;

         dist[i][j] = cost[i][j];

      }

   for (k = 0; k < n; k++)

      for (i = 0; i < n; i++)

         for (j = 0; j < n; j++)

            if (dist[i][j] > cost[i][k] + dist[k][j])

               dist[i][j] = cost[i][k] + dist[k][j];
```

```
    for (i = 0; i < n; i++) {
        printf("\nRouting table for Node %d:\n", i);
        printf("Destination\tDelay\n");
        for (j = 0; j < n; j++) {
            if (dist[i][j] >= INF)
                printf("%d\t\tUnreachable\n", j);
            else
                printf("%d\t\t%d\n", j, dist[i][j]);
        }
    }
    return 0;
}
```

Output:

```
Enter number of nodes: 3
Enter cost matrix (0 for no link):
0 2 7
2 0 3
7 3 0

Routing table for Node 0:
Destination Delay
0       0
1       2
2       5

Routing table for Node 1:
Destination Delay
0       2
1       0
2       3
```

```
Routing table for Node 2:
Destination Delay
0       5
1       3
2       0
```

EXPERIMENT-12

Develop a program to implement Broadcast tree by taking subnet of hosts.

Description:

A broadcast tree is used to distribute data from a source host to all reachable hosts in a network. The program represents the subnet using an adjacency matrix and uses Breadth-First Search (BFS) to construct the broadcast tree. It visits each host only once and records the links used to reach new hosts. This avoids duplicate transmissions along the tree and helps deliver broadcast messages efficiently.

Program

```
#include <stdio.h>

int main() {
    int n, i, j, root;
    printf("Enter number of hosts: ");
    scanf("%d", &n)
    int a[n][n];
    printf("Enter adjacency matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &a[i][j]);
    printf("Enter source host (0 to %d): ", n - 1);
    scanf("%d", &root);
    int visited[n], queue[n];
    for (i = 0; i < n; i++)
        visited[i] = 0;
    int front = 0, rear = 0;
    queue[rear++] = root;
    visited[root] = 1;
    printf("\nBroadcast Tree Edges:\n");
    while (front < rear) {
        int u = queue[front++];
```

```
        for (i = 0; i < n; i++) {
            if (a[u][i] && !visited[i]) {
                printf("%d -> %d\n", u, i);
                visited[i] = 1;
                queue[rear++] = i;
            }
        }
    }
    return 0;
}
```

Output:

```
Enter number of hosts: 4
Enter adjacency matrix:
0 1 1 0
1 0 0 1
1 0 0 1
0 1 1 0
Enter source host (0 to 3): 0

Broadcast Tree Edges:
0 -> 1
0 -> 2
1 -> 3
```

EXPERIMENT – 13

Wireshark

i. **Starting Wireshark**
ii. **Packet Capture using Wireshark**
iii. **Viewing Captured Traffic**
iv. **Analysis and Statistics & filters**

Starting Wireshark

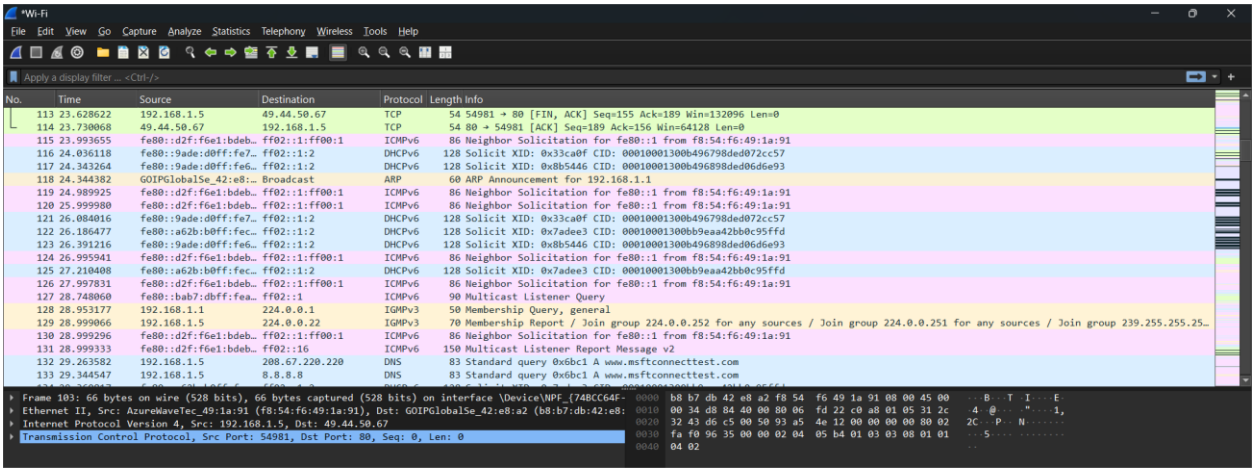
- Wireshark is a network protocol analyzer that captures and displays packets in real time.
- The Wireshark application can be opened from the start menu or desktop shortcut.
- On launch, it shows a list of available network interfaces.
- The desired interface can be chosen based on where the traffic is to be monitored.
- Optionally, capture filters can be applied before starting to capture to narrow down the type of packets collected.

Packet Capture Using Wireshark

- To capture the packets, correct network interface should be selected i.e ones that have wave-like symbol.
- The wave-like icon indicates that wireshark is actively detecting traffic on that interface in real time. (eg, WiFi, Ethernet)
- Then click on the Start Capturing Packets button(shaped like a shark fin).
- Wireshark begins capturing all traffic passing through the selected interface.

Viewing Captured Traffic

I selected the Wi-Fi interface from the list of available network interfaces. The following packets were captured as they were transmitted to and from my system through this interface.



192.168.1.5 - This is the local IP address of my system.

The IP addresses mentioned above correspond to the following
49.44.50.67 – akamai.net
8.8.8.8 - dns.google

224.0.0.251 – mdns.mcast.net
34.120.208.123 – mozilla.com

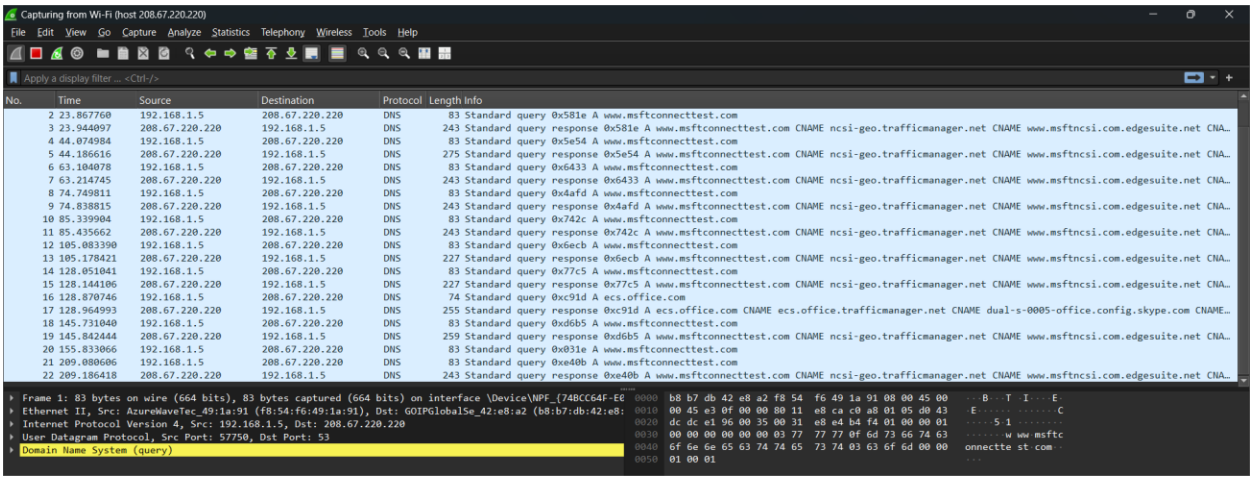
Filters

Capture Filters

A capture filter in Wireshark is used to limit the packets from the network based on specific criteria.

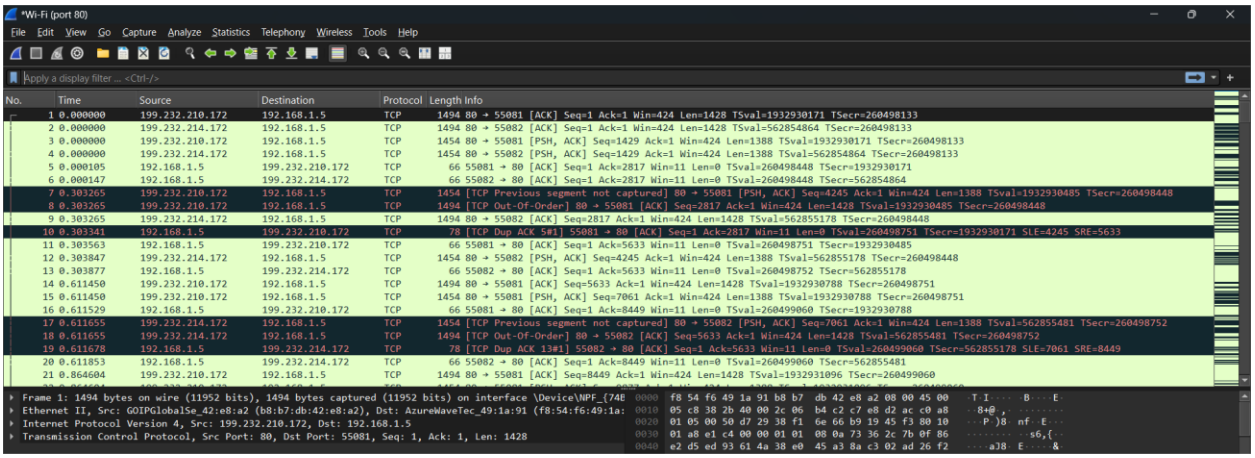
1. host 208.67.220.220

This captured all the packets where the source or destination IP is 208.67.220.220



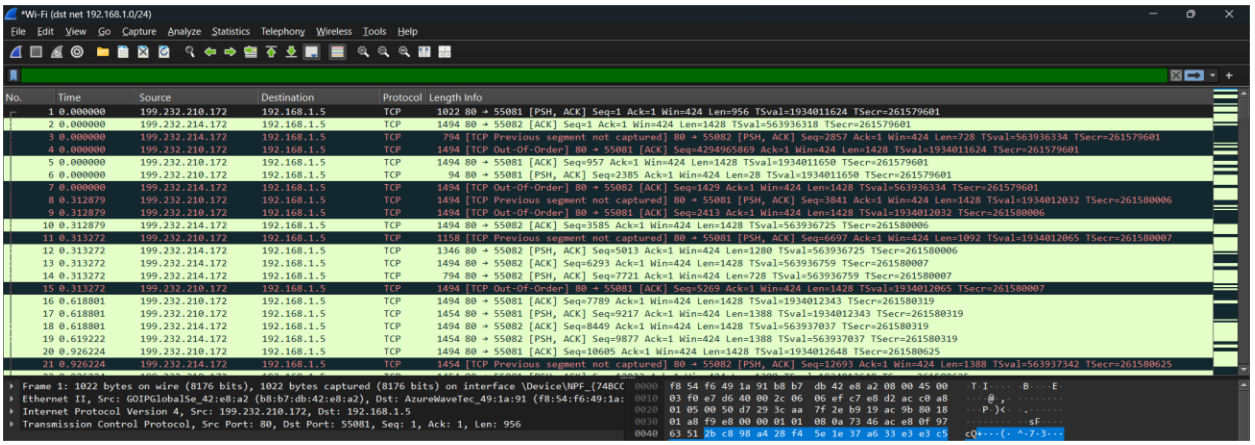
2. port 80

This captured only packets that use port 80 as either source or destination



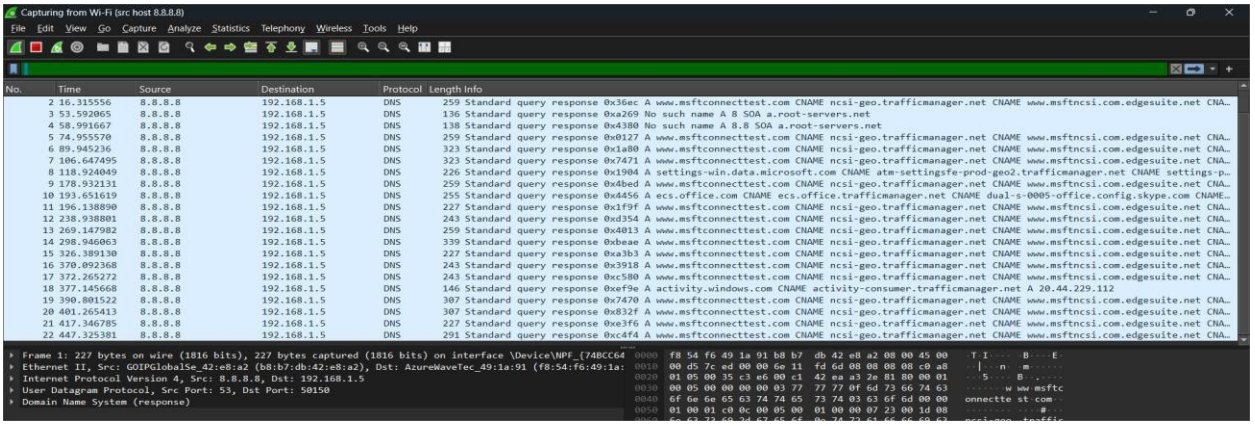
3. dst net 192.168.1.0/24

This captured the packets going to any IP in the 192.168.1.0 – 192.168.1.255 range



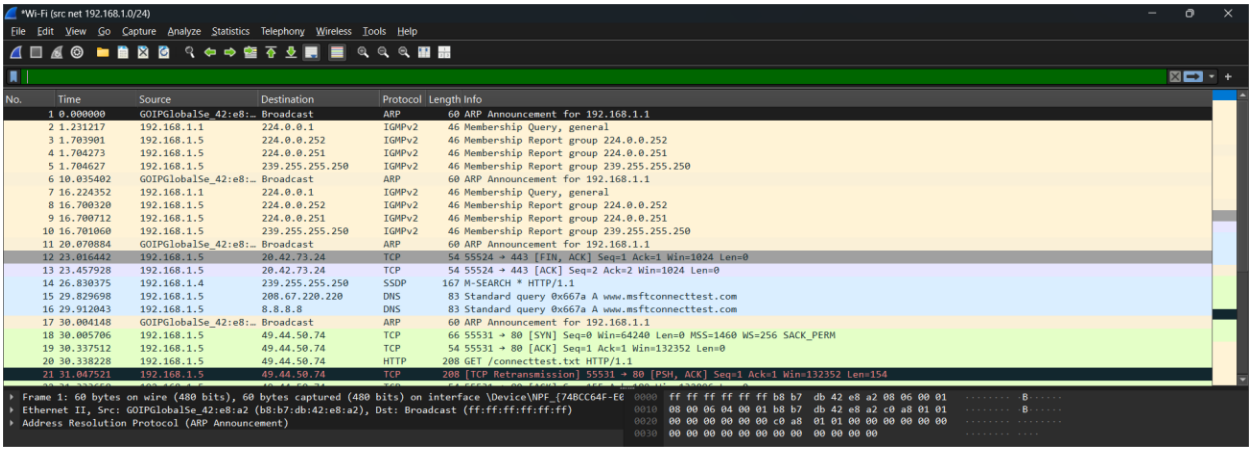
4. src host 8.8.8.8

It captured the packets sent from 8.8.8.8, i.e where the source ip address matched 8.8.8.8



5. src net 192.168.1.0/24

This captured the packets coming from any IP in the 192.168.0.0 – 192.168.1.255 range.

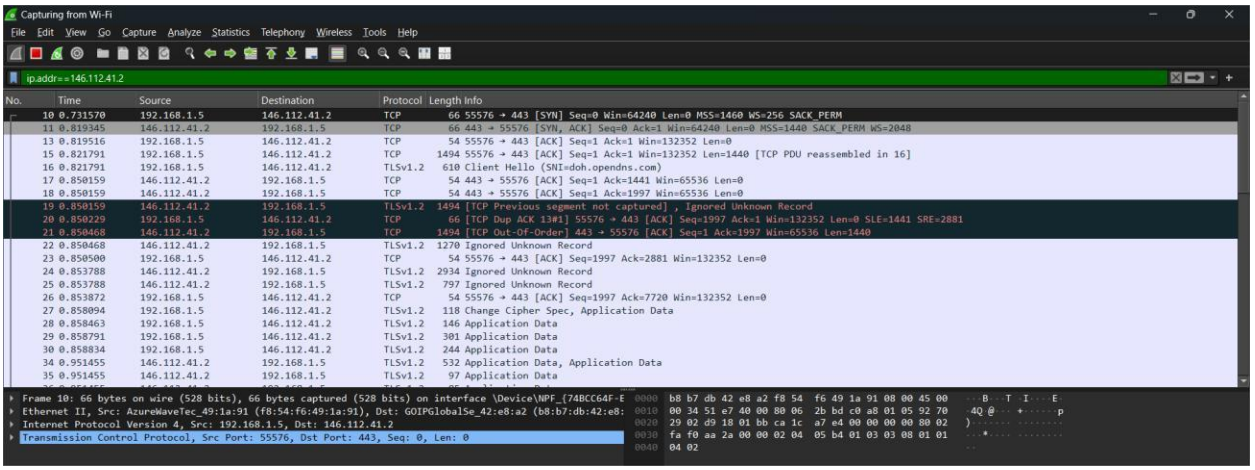


Display Filters

A display filter is used to filter and view specific packets from the captured data after the capture has taken place.

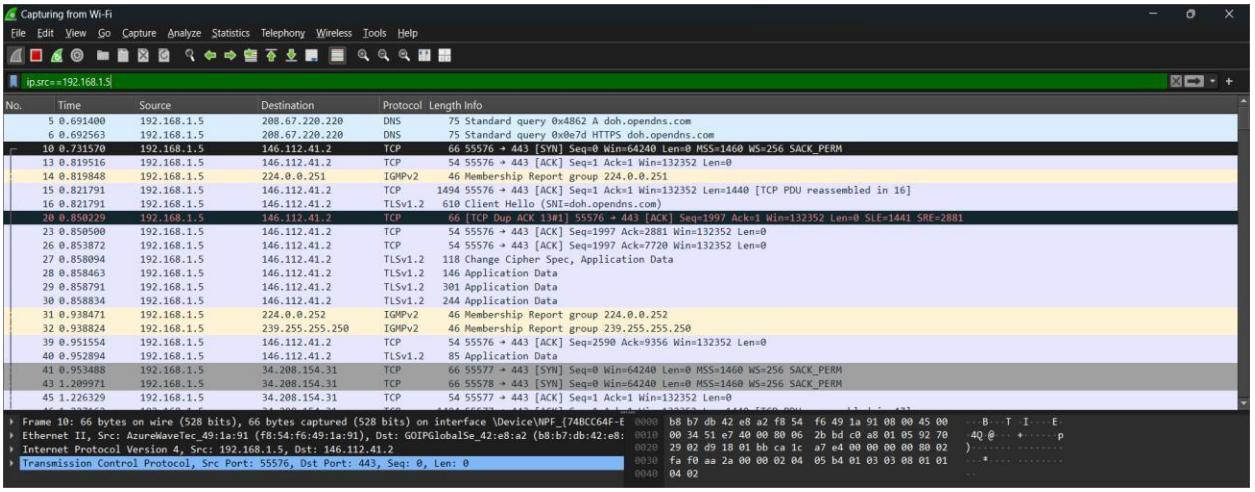
1. ip.addr == 146.112.41.2

It captured only the packets that are either sent/ received from the IP address 146.112.41.2



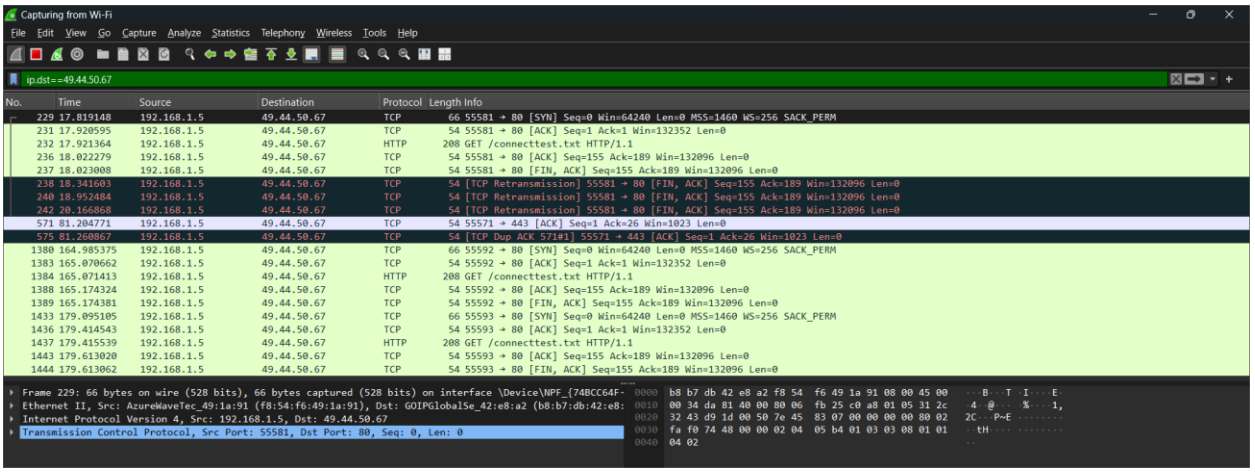
2. ip.src == 192.168.1.5

It displayed only packets sent from 192.168.1.5 after they have been captured.



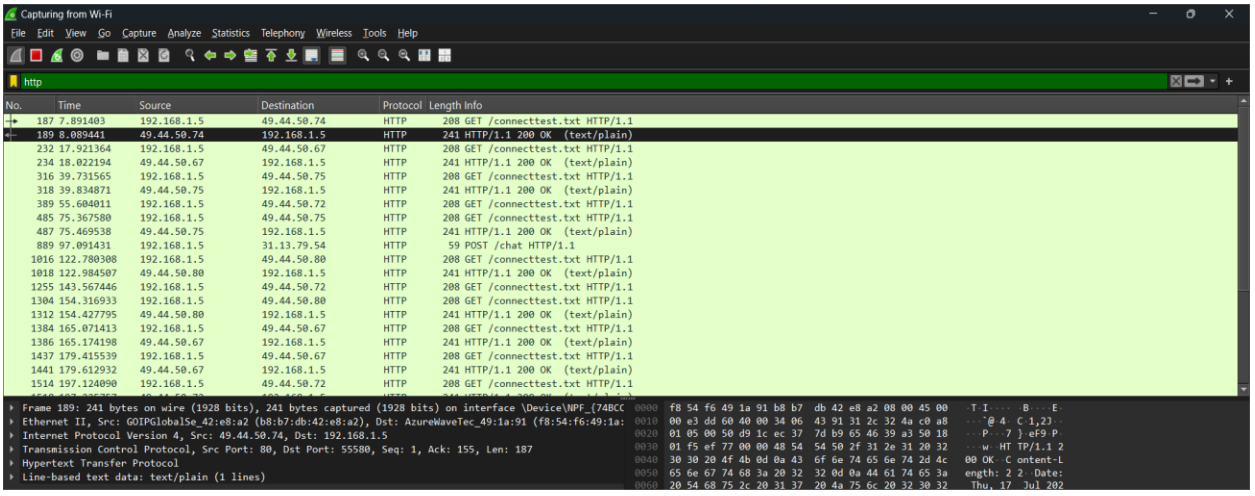
3. ip.dst == 49.44.50.67

It showed packets with destination address 49.44.50.67



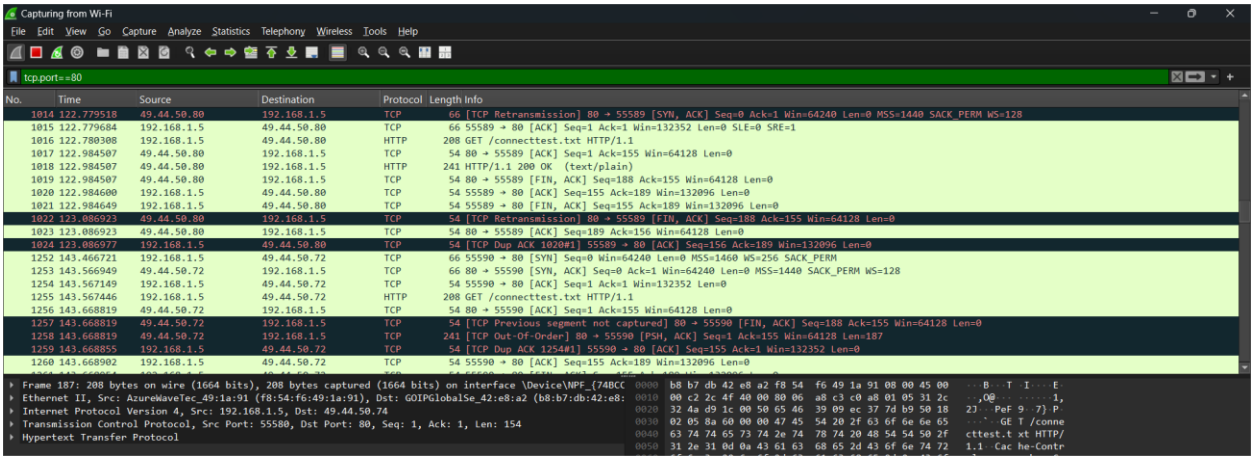
4. http

It has shown only http packets.



5. tcp.port == 80

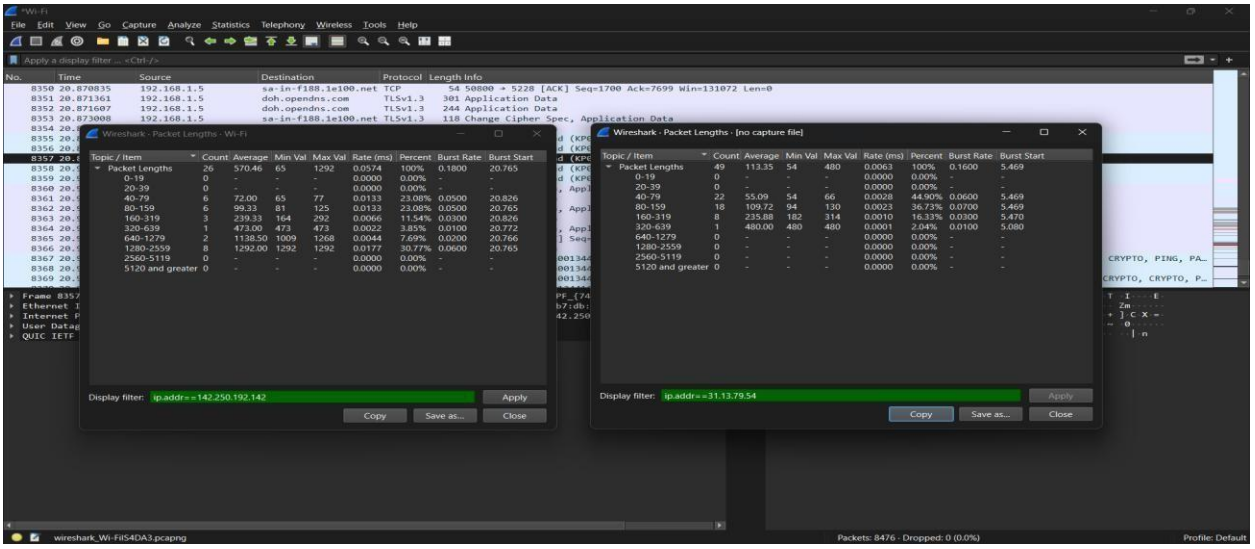
It has displayed all the packets where the TCP source or destination port is 80.



Analysis & Statistics

I have compared two different services to analyze the packet sizes and distribution of network traffic across them.

IP: 142.250.192.142 (YouTube) IP: 31.13.79.54 (Whatsapp)



- YouTube had a mix of packet sizes, but leaned heavily on large packets.
    - 2 packets in the 640-1279 bytes range.
    - 8 packets in the 1280-2554 bytes range.
- Whats App used smaller packets
    - 22 packets in the 40-79 bytes range
    - 18 packets in the 80-159 bytes range
- The burst time for Whatsapp was around 5.469 seconds, whereas for YouTube it was 20.756 seconds, which indicates WhatsApp began transmitting data carly.
- Whatsapp had more packets per second but with smaller sizes and YouTube had larger packets, i.e less packets per second but with larger sizes
- Therefore, WhatsApp involved frequent small packets, suitable for text messaging and You Tube sent fewer but much larger packets, suitable for continuous video streaming.

Flow graphs:

To understand the direction and sequence of communication between my system and the services, I analyzed the flow graphs too for youtube and Whatsapp.

YouTube (IP: 142.250.192.142)

Incoming Packets: 14

Outgoing Packets: 12

Total: 26, Ratio : 12/14 = 0.85

WhatsApp (IP: 31.13.79.34)

Incoming packets: 28

Outgoing packets: 21

Total:49, Ratio: 21/28 = 0.75

EXPERIMENT-14

Discover active hosts, open ports and running services in a network using Nmap

Nmap

- Nmap is a network scanning tool used to discover hosts and services on a computer network.
- It helps in network mapping, port scanning and security auditing.

Basic Scanning Commands

1. nmap <target>

This is the most basic command which targets the target (IP address Or hostname) using default settings. St detects open ports and services that respond.

Example: nmap 192.168.1.1

2. nmap -p <port>

This scans a specific port on the target system instead of the default ports. It is useful when is running. we want to check if a certain service

Example: nmap -p 80 192.168.1.1

3. **nmap -p <range>**

This scans a custom range of ports instead of all or default ports. st's faster and more focused, ideal when you know the potential vulnerable ports.

Example: nmap -p 20-100 192.168.1.1

4. **nmap -p-**

This scans all 65,535 TCP ports on the target System. It is ver thorough and often used when performing full port enumeration.

Example: nmap -p- 192.168.1.1

Scan Techniques

1. nmap –sS (TCP SYN scan)

This is a stealthy scan that sends SYN packets and analyzes responses without completing the full TCP handshake. It is fast and commonly used for discovering open ports discreetly**.**

Example: nmap –sS 192.168.1.1

2. nmap –sT (TCP Connect scan)

This scan completes the hall TCP connection with the target, making it less stealthy but more reliable on systems where SYN scan requires privileges.

Example: nmap -sT 192.168.1.1

3. nmap-sU (UDP scan)
This scan checks for open UDP ports, which are harder to detect than TCP ports due to lack of response behaviour. It is uschul for finding Services like DNS, SNMP, TFTP.

Example: nmap -sU 192.168-1-1

4. nmap –sN (Null Scan)
This sends packets with no TCP flags set and analyze responses. Some firewalls and intrusion detection systems may overlook this type of scan.

Example: nmap -sN 192.168.1.1

Service and Version Detection

1. nmap -SV
This enables version detection for services running on open ports. It helps identify specific software and versions, which is useful for vulnerability assessment.

Example: nmap -SV 192.168.1.1

2. nmap --version - all
This Command forces Nmap to try all available version detection method providing the most comprehensive output. It is used when maximum service detail is needed.

Example: nmap --versionall 192.168.1.1

Command to detect active hosts

nmap –sn <target>

Performs a ping scan to determine which hosts are up without scanning ports.

Example; nmap –sn 10.10.12.0/24



Commands to detect open ports and running services

1. nmap –sV <target>

The –sV option in Nmap enables version detection , which not only identifies open ports but also determines the specific services and their versions running on these ports.

Example: nmap –sV 10.10.12.161

```
C:\Users\HP>nmap -sV 10.10.12.161
Starting Nmap 7.97 ( https://nmap.org ) at 2025-07-24 13:54 +0530
Nmap scan report for 10.10.12.161
Host is up (0.0013s latency).
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE       VERSION
135/tcp   open  msrpc         Microsoft Windows RPC
139/tcp   open  netbios-ssn   Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds?
50006/tcp open  msrpc         Microsoft Windows RPC
MAC Address: 24:6A:0E:7A:59:51 (HP)
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 60.83 seconds

C:\Users\HP>
```

2. nmap –p- <target>

It scans all the TCP ports in the target system. **Example:** nmap –p- 10.10.12.1

```
C:\Users\HP>nmap  -p- 10.10.12.161
Starting Nmap 7.97 ( https://nmap.org ) at 2025-07-30 13:55 +0530
Nmap scan report for 10.10.12.161
Host is up (0.00026s latency).
Not shown: 65523 closed tcp ports (reset)
PORT      STATE SERVICE
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
5040/tcp  open  unknown
7680/tcp  open  pando-pub
49664/tcp open  unknown
49665/tcp open  unknown
49666/tcp open  unknown
49667/tcp open  unknown
49668/tcp open  unknown
49670/tcp open  unknown
49671/tcp open  unknown
MAC Address: 24:6A:0E:7A:59:51 (HP)

Nmap done: 1 IP address (1 host up) scanned in 10.72 seconds

C:\Users\HP>
```

EXPERIMENT-15

Find Operating System in a host using Nmap

1. nmap –O <target>

This command enables os detection using TCP/IP Stack fingerprinting. It Sends a series of crafted packets and analyzes responses to determine the OS type and version

Example: nmap -O 192.168.1.1

2. nmap -A <target>

The - A option enables aggressive scan which includes OS detection, script scanning and traceroute

Example: nmap - A scanme.nmap.org

3-nmap -O -v <target>

Adds -v for verbose output, showing more details about the os guess and the logic behind it.

Example: nmap -0 -v 192.168.11

4. nmap -O --osscan-guess <target>

The --Osscan-guess option allows Nmap to guess the os more aggressively when it's unsure. It's helpful when exact detection fails, but Nmap can still make an educated guess.

Example: nmap -O --osscan-guess 192.168.1.1

Commands to detect Operating System

1. nmap –O 10.10.12.161

```
C:\Users\HP>nmap -O 10.10.12.161
Starting Nmap 7.97 ( https://nmap.org ) at 2025-07-30 13:54 +0530
Nmap scan report for 10.10.12.161
Host is up (0.0033s latency).
Not shown: 997 closed tcp ports (reset)
PORT    STATE SERVICE
135/tcp open  msrpc
139/tcp open  netbios-ssn
445/tcp open  microsoft-ds
MAC Address: 24:6A:0E:7A:59:51 (HP)
Device type: general purpose
Running: Microsoft Windows 10|11
OS CPE: cpe:/o:microsoft:windows_10 cpe:/o:microsoft:windows_11
OS details: Microsoft Windows 10 1703 or Windows 11 21H2
Network Distance: 1 hop

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 2.34 seconds

C:\Users\HP>
```

2. nmap -O- –sV <target>

This command scans the host, attempts to identify operating system and detects service versions running on open ports

Example: nmap –O- –sV 10.10.12.161

```
C:\Users\HP>nmap -o- -sV 10.10.12.161
# Nmap 7.97 scan initiated Thu Jul 24 13:52:05 2025 as: nmap -o- -sV 10.10.12.161
Nmap scan report for 10.10.12.161
Host is up (0.0018s latency).
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE       VERSION
135/tcp   open  msrpc         Microsoft Windows RPC
139/tcp   open  netbios-ssn   Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds?
50006/tcp open  msrpc         Microsoft Windows RPC
MAC Address: 24:6A:0E:7A:59:51 (HP)
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
# Nmap done at Thu Jul 24 13:53:06 2025 -- 1 IP address (1 host up) scanned in 60.89 seconds
```

EXPERIMENT-16

16. Do the following by using NS2 Simulator

i. NS2 Simulator-Introduction

ii. Simulate to Find the Number of Packets Dropped

iii. Simulate to Find the Number of Packets Dropped by TCP/UDP

iv. Simulate to Find the Number of Packets Dropped due to Congestion

v. Simulate to compare Data Rate & Throughput

(i) NS2 Simulator-Introduction

NS-2 (Network Simulator 2) is an open-source, discrete-event network simulation tool used to design, simulate, and analyze computer networks. It is widely used in Computer Networks (CN) to study the behavior and performance of different network protocols without requiring a real physical network.

NS-2 supports the simulation of both wired and wireless networks. It provides facilities to create network topologies, configure nodes, establish connections, generate network traffic, and analyze network performance.

NS-2 mainly uses OTcl (Object-oriented Tool Command Language) scripts to describe the network and its simulation parameters. The core simulation functionality is implemented in C++, while OTcl is used to configure and control the simulation.

The simulation results are generally stored in trace files, which can be analyzed to measure parameters such as throughput, packet loss, delay, and network performance. NS-2 can also generate NAM (Network Animator) files to visualize the movement of packets and the behavior of network nodes.

(ii),(iii) & (iv)

We need to run the program in the linux environment.we need to install ubuntu in our pc ie,windows.After the completion of the downloading and the installing of the ubuntu...install NS2 Simulator.

After the Completion of the installation process of NS2 Simulator.Create a folder for the simulation Files.

Open Ubuntu terminal and run the following commands as shown below:

Mkdir ns2work click enter

cd ns2work click enter

Now,To create the Simulation file,type:

nano congestion.tcl-->this opens the nano text editor,where you can write the NS2 Simulator program.

Open the File by using:

nano congestion.tcl

nano congestion.tcl

```
set ns [new Simulator]
set tr [open congestion.tr w]
$ns trace-all $tr
proc finish {} {
   global ns tr
   $ns flush-trace
   close $tr
   exit 0
}
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
set n4 [$ns node]
$ns duplex-link $n0 $n1 10Mb 10ms DropTail
$ns duplex-link $n4 $n1 10Mb 10ms DropTail
$ns duplex-link $n1 $n2 1Mb 20ms DropTail
$ns duplex-link $n2 $n3 10Mb 10ms DropTail
$ns queue-limit $n1 $n2 10
set tcp [new Agent/TCP/Newreno]
$tcp set fid_ 1
$ns attach-agent $n0 $tcp
set sink [new Agent/TCPSink]
$ns attach-agent $n3 $sink
$ns connect $tcp $sink
set ftp [new Application/FTP]
$ftp attach-agent $tcp
set udp [new Agent/UDP]
$udp set fid_ 2
```

$ns attach-agent $n4 $udp

set null [new Agent/Null]

$ns attach-agent $n3 $null

$ns connect $udp $null

set cbr [new Application/Traffic/CBR]

$cbr attach-agent $udp

$cbr set packetSize_ 1000

$cbr set interval_ 0.002

$cbr set random_ false

$ns at 0.5 "$ftp start"

$ns at 0.6 "$cbr start"

$ns at 9.5 "$ftp stop"

$ns at 9.5 "$cbr stop"

$ns at 10.0 "finish"

$ns run

Saving the File and exiting from the nano editor:

CTRL+O,Enter and CTRL+X.

Now,run the command:

ns congestion.tcl

If you see a message related to tclsh,it is generally only a warning and doesnot neceesarily mean that the simulation failed.

To check whether the simulation was sucessful,run:

ls

if the file:

congestion.tr is created,The NS2 Simulaton has run successfully.

Now,Create the Result Analysis File(result.awk)

Run the following command in the Ubuntu terminal:

nano result.awk

Then,this will open the nano text editor and creates a new file named as **result.awk**

nano result.awk

```
$1=="d"&&$5=="tcp" { tcpDrop++ }
$1=="d"&&($5=="cbr" || $5=="udp") { udpDrop++ }
$1=="d" && $3==1 && $4==2 { congestionDrop++ }
$1=="r"&&$4==3&&$5=="tcp" {
   tcpBytes += $6
   if (!tcpSeen++) tcpFirst=$2
   tcpLast=$2
}
$1=="r"&&$4==3&&($5=="cbr" || $5=="udp") {
   udpBytes += $6
   if (!udpSeen++) udpFirst=$2
   udpLast=$2
}
END {
   print "TCP dropped packets =", tcpDrop
   print "UDP dropped packets =", udpDrop
   print "Congestion dropped packets =", congestionDrop
   printf "TCP throughput = %.3f Mbps\n", (tcpBytes*8/(tcpLast-tcpFirst))/1000000
   printf "UDP throughput = %.3f Mbps\n", (udpBytes*8/(udpLast-udpFirst))/1000000
   print "UDP configured data rate = 4 Mbps"
}
```

Saving the File and exiting from the nano editor:

CTRL+O,Enter and CTRL+X.

If you cant find the result.awk.file then run:

ls

Now,view the Final results:-

Run the following Command:

awk -f result.awk congestion.tr

This command executes the result.awk program on the congestion.tr trace file.

The terminal will display the calculated the simulation results,such as:

OUTPUT:

